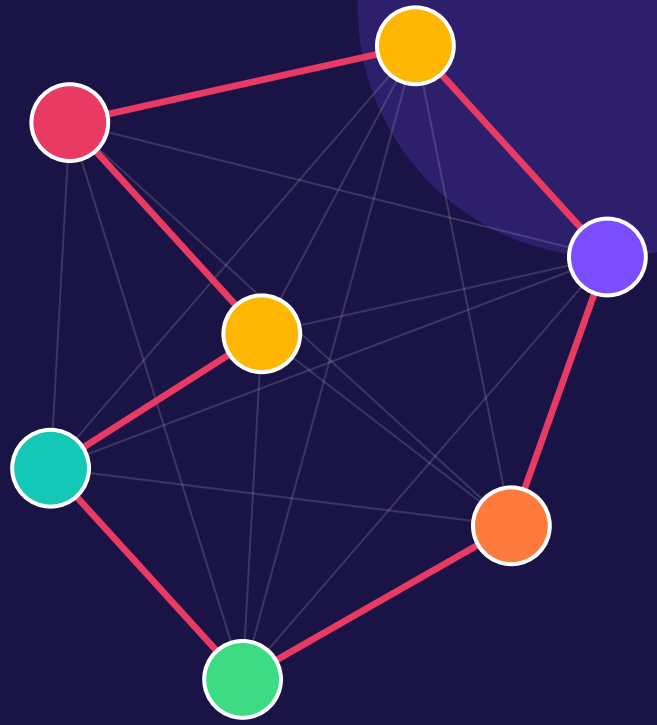
DAA • CA3 • USE OF AI

Branch and Bound: Traveling Salesman Problem



Arshiya Kayarkar

PRN: 25070521119

Section B

Generated using Claude

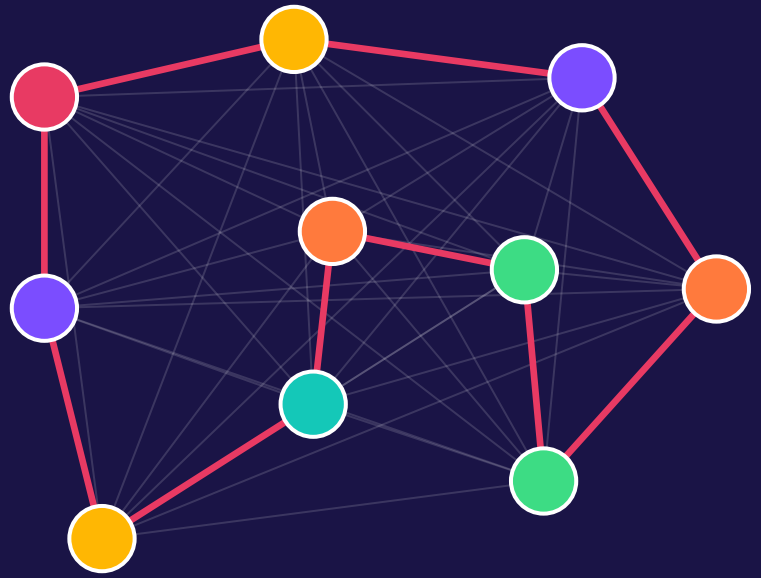
1 • THE HOOK

Ten cities. 181,440 routes.

Which one's shortest?

(10 − 1)! ÷ 2 = 181,440 possible tours

Brute force takes forever.



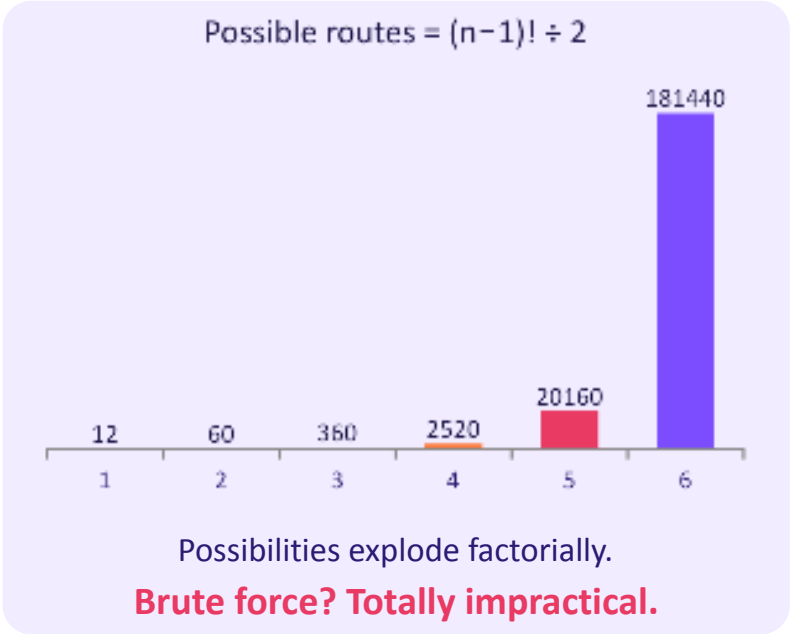
Generated using Claude

The Traveling Salesman Problem

Visit every city once
Each stop exactly one time

Return home
Minimize the total distance

It's NP-Hard
No known fast exact method



Possibilities explode factorially.

Brute force? Totally impractical.

Enter Branch and Bound



BRANCH

Split into subproblems



BOUND

Estimate minimum cost with cost matrix reduction



PRUNE

Bound exceeds best? Cut the branch!

Always expand the lowest bound. Skip hopeless branches. Optimal tour, fewer checks. **Worst case: still exponential.**

Cost Matrix Reduction

Original matrix

∞	10	15	20
5	∞	9	10
6	13	∞	12
8	8	9	∞

rows −29

After row reduction

∞	**0**	5	10
0	∞	4	5
0	7	∞	6
0	**0**	1	∞

cols −6

After column reduction

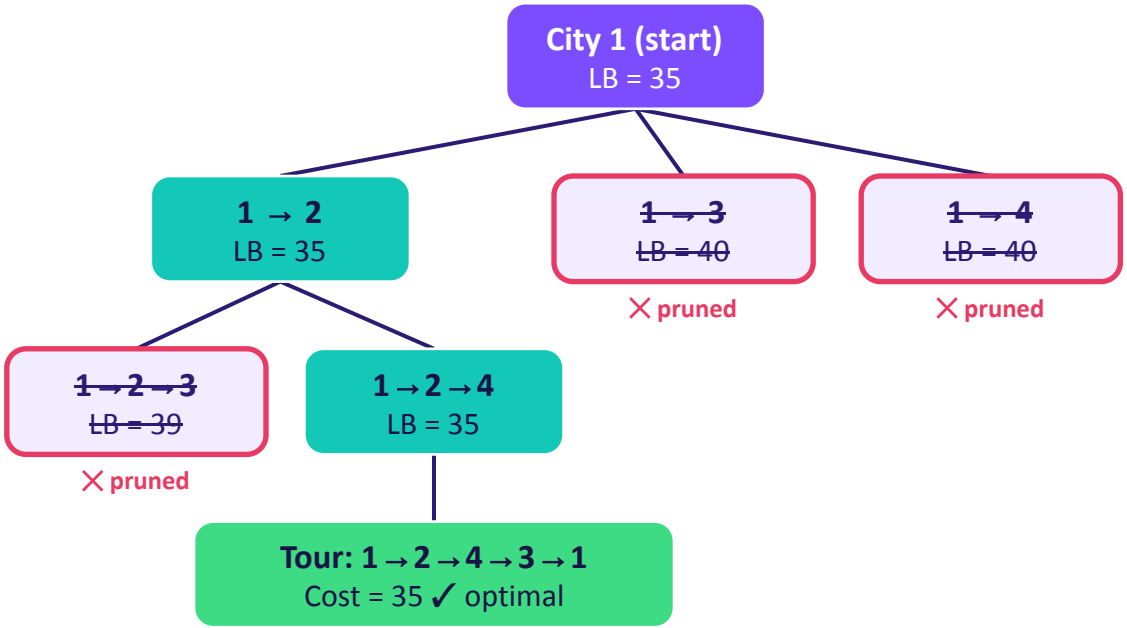
∞	**0**	4	5
0	∞	3	**0**
0	7	∞	1
0	**0**	**0**	∞

Row minima
10 + 5 + 6 + 8 = 29

Column minima
0 + 0 + 1 + 5 = 6

Lower bound
29 + 6 = 35

Pruning the Tree



Expand lowest bound
LB 35 beats 40 and 40

Best tour found: 35
1 → 2 → 4 → 3 → 1

Prune the rest
39 and 40 exceed 35

Algorithm Steps

1. **Reduce the root matrix**
   Gives the first lower bound
2. **Branch**
   One child for each unvisited city
3. **Bound each child**
   Reduce its matrix and compute its LB
4. **Expand the lowest LB**
   Use a priority queue (min-heap)
5. **Prune and stop**
   Drop LB ≥ best; stop at best full tour

CHILD BOUND

LB(child) = LB(parent) + cost(i → j) + reduction

e.g. 1 → 3: 35 + 4 + 1 = 40

How Do the Methods Compare?

Brute Force

O(n!)

Checks every possible tour

Dynamic Programming

$O(n^2 \cdot 2^n)$

Held-Karp: exact, but needs $O(n \cdot 2^n)$ memory

Branch and Bound

Worst case O(n!)

Pruning makes it much faster in practice

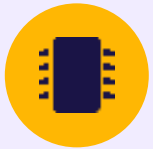
Branch and Bound is exact. It always returns the optimal tour; its speed depends on how tight the lower bound is.

Real-World Applications



Delivery and logistics

Plan routes for vans and couriers



Circuit board drilling

Shortest path for the drill head



DNA sequencing

Order genome fragments efficiently



Warehouse robots

Pick items with minimal travel

5 • CALL TO ACTION

Conclusion and Next Steps



Try it on paper

Reduce a 4×4 matrix by hand



Code it in C++

Priority queue + lower bounds



Check my GitHub repo

https://github.com/Arshiya478/DAA-Lab-Sem-III



Key takeaway

Bound smart. Prune early.